06 · Multicore-Echtzeit

AMP, Shared Memory, Cache und MPU · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 55

Willkommen in der Multicore-Königsklasse!

- Sechster Termin: zwei Kerne, ein Chip, keine automatische Isolation
- Labor: **M7 Bare-Metal**, **M4 FreeRTOS** (CMSIS-RTOS2)
- HSEM kommt hier als **Boot-Handshake** vor, wenn das Template ihn erzeugt
- HSEM als Laufzeitsperre bleibt Termin 8
- Nächstes Mal: Tracing, nicht die volle HSEM-Vorlesung

Notizen 1 / 55

Zwei Kerne auf einem Chip rechnen nicht automatisch isoliert. Im Lehrbild regelt der Cortex-M4 einen Motor, der Cortex-M7 bedient Anzeige oder Kommunikation. Das ist eine Aufgabenteilung, keine Messung. Bare Metal heißt: auf dem M7 läuft kein RTOS-Kernel. FreeRTOS ist das Echtzeitbetriebssystem auf dem M4. CMSIS-RTOS2, Common Microcontroller Software Interface Standard in Version 2, ist die Schnittstelle zu diesem Kernel, nicht ein Scheduler über beide Kerne. HSEM, Hardware Semaphore, dient hier als Boot-Benachrichtigung, wenn das Template sie erzeugt. Als Laufzeitsperre kommt sie in Termin 8. Tracing folgt in Termin 7.
Drei Ziele: gemeinsame Ressourcen erkennen, dasselbe Layout in beiden Programmen vereinbaren und eine Datenübergabe mit Reihenfolge und Besitz erklären. Eine fehlende `.ioc` heißt: Takte und Boardnamen stehen im generierten Cube-Projekt, nicht in dieser Folie.
Dafür betrachten wir Architektur, Hardware und Kommunikation nacheinander.

Folie 2 / 55

Agenda für heute

1. Ein Kern, SMP, AMP
2. H755: Domänen, Bus, Boot
3. Zwei Images, ein SRAM4-Block
4. Cache, Ordnung, Ownership
5. MPU und Ping-Pong

Notizen 2 / 55

Die fünf Blöcke hängen voneinander ab. Ohne die Unterscheidung SMP, Symmetric Multiprocessing, und AMP, Asymmetric Multiprocessing, bleibt unklar, wer die Aufgaben plant. Ohne die Hardwarekarte bleibt unklar, welche Speicherwege geteilt sind. Ohne dasselbe Layout zeigen zwei Programme trotz gleichen Namens verschiedene Variablen. Ohne Cache- und Ordnungsmodell kommt die Übergabe nicht zuverlässig an. MPU, Memory Protection Unit, legt später fest, wie der M7 den gemeinsamen Bereich behandelt.
Aus Vorlesung 2: EDF, Earliest Deadline First, wählt nach der frühesten Deadline. Die Auslastung ist $U = \sum C_i/T_i$ mit Ausführungsbedarf C, Periode T und relativer Deadline D. $U \leq 1$ gilt im idealisierten Einprozessor-Modell mit unabhängigen, präemptiven Aufgaben und $D = T$. Versteckte Kosten dürfen nicht fehlen. Zwei Kerne übernehmen diese Formel nicht.
Leitfrage: Wie übergibt der M7 dem M4 einen Wert, den der M4 wirklich sieht? Am Ende soll die Folge 1, 3, 5 das beantworten.
Warum stößt ein einzelner Kern zunächst an Grenzen?

Folie 3 / 55

Recap: Die Grenzen des Single-Core

Didaktische Aufteilung, keine Messung:

Arbeit	ein Kern	Laboridee
Regelung	teilt sich die Zeit	eher M4
Netz, GUI	dieselbe CPU	eher M7

Gemeinsames SRAM, die Busbrücke und DMA bleiben Störquellen.
Vereinfacht, nur dynamische Schaltleistung:

$$P_{\text{dyn}} \approx \alpha\, C\, V^2\, f$$

Spannung, Aktivität und Leckstrom kommen dazu. Keine gemessene Kurve.

Notizen 3 / 55

Auf einem Kern werden gleichzeitig aktive Anforderungen zeitlich verschachtelt. Die Tabelle ist eine mögliche Aufgabenteilung, keine Leistungsmessung. CPU ist der Prozessor, GUI die grafische Bedienung, DMA der direkte Speicherzugriff durch Hardware, IRQ die Interruptanforderung, OS das Betriebssystem. Regelung und Netz teilen sich dieselbe CPU, bis ein zweiter Kern eine der Arbeiten übernimmt. Gemeinsames SRAM, die Brücke und DMA bleiben Störquellen.
Die dynamische Schaltleistung ist näherungsweise $P_{\text{dyn}} \approx \alpha C V^2 f$. Hier ist C eine wirksame Kapazität, nicht der Ausführungsbedarf aus dem Scheduling. α ist der Aktivitätsfaktor, V die Spannung, f die Frequenz. Leckstrom und weitere Kosten kommen dazu. Es gibt keine gemessene Kurve. $U = 1$ im Idealmodell ist keine Reserve für bisher vergessene Arbeit.
Frage: Welche Kosten bleiben mit einem zweiten Kern? Interferenz auf gemeinsamen Wegen, Kopien und das Protokoll. Der zweite Kern beseitigt sie nicht.
Eine höhere Frequenz hilft bei manchen Engpässen, hat aber Grenzen.

Folie 4 / 55

Erhöhung der Taktfrequenz?

- Höheres f erhöht P_{dyn}, nicht „exponentiell“ als einziges Modell
- 480 MHz (M7) und 240 MHz (M4) sind spezifizierte Maxima
- 400/200 MHz nur, wenn der Clock-Tree, VOS und die Flash-Wartezeit das hergeben
- Mehr Takt parallelisiert keine zwei Aufgaben auf einem Kern

Notizen 4 / 55

Ein Megahertz sind eine Million Taktzyklen je Sekunde. Ein Taktzyklus ist nicht schon eine fertige Operation. Im vereinfachten Modell $P_{\text{dyn}} \approx \alpha C V^2 f$ verdoppelt eine Verdopplung von f bei konstantem V und α die dynamische Leistung. Ändert sich zugleich die Spannung, wirkt ihr Quadrat zusätzlich. VOS ist die herstellerspezifische Stufe der internen Spannungsskalierung. Flash-Wartezyklen gleichen die Speicherzugriffszeit an den schnelleren Kern an. 480 MHz für den M7 und 240 MHz für den M4 sind spezifizierte Höchstwerte, keine schon geprüfte Laborwahl. Ob 400 und 200 MHz legal sind, steht im Clock-Tree des Projekts, nicht auf dieser Folie.
Mehr Takt verschiebt den Engpass, er teilt die Arbeit nicht auf zwei Kerne.
Parallelität verändert die Architektur, nicht nur den Takt.

Folie 5 / 55

Der Ausweg: Multicore-Architekturen

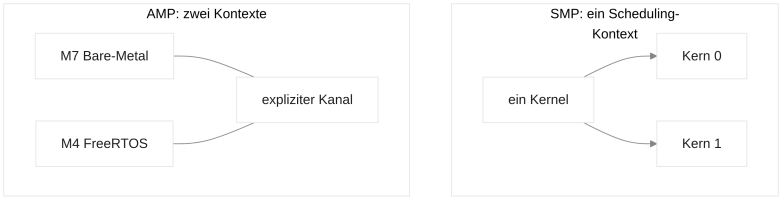
- Zwei Kerne können rechnen, solange ihre Datenwege frei sind
- Partitionieren heißt: getrennte Firmware und klare Zuständigkeit
- Es heißt nicht: der andere Kern ist zeitlich unsichtbar

Notizen 5 / 55

Zwei Kerne können unabhängige Rechenabschnitte nebeneinander ausführen und danach Daten tauschen. Der Tausch und jeder Zugriff auf dieselbe Speicherbank begrenzen den Gewinn. Partitionierung heißt: Software und Ressourcen sind fest zugeordnet. Zeitliche Isolation hieße, die gegenseitige Störung sei begrenzt. Getrennte Firmware allein liefert diesen Nachweis nicht. Ein konkreter Weg: beide Kerne lesen dieselbe SRAM-Bank, die Busmatrix serialisiert den Konflikt. Eine Speedup-Zahl wird hier nicht behauptet. AMP, die asymmetrische Aufteilung, erleichtert die Zuständigkeit. Sie ist kein Sicherheitsnachweis.
Es gibt verschiedene Organisationsformen. Der nächste Vergleich führt sie ein.

Folie 6 / 55

Multiprocessing-Architekturen



- SMP: gemeinsamer OS-Kontext, nicht „Kerne müssen identisch und gleichtaktig sein“
- Ready-Queues dürfen global oder je Kern sein, plus Affinität
- Migration verschiebt einen gesicherten Kontext; der Zielcache ist nicht leer versprochen

Notizen 6 / 55

Links plant ein Betriebssystemkontext Aufgaben für mehrere geeignete Kerne. Die Bereiten können in einer gemeinsamen oder in kernbezogenen Warteschlangen stehen. Das ist SMP, Symmetric Multiprocessing. Rechts laufen getrennte Firmwarekontexte. Das ist AMP, Asymmetric Multiprocessing. OS ist das Betriebssystem. Ready heißt ausführungsbereit. Affinität beschränkt, auf welchen Kernen eine Aufgabe laufen darf. Gleiche Taktfrequenz ist keine SMP-Bedingung. Die Hardware und der Kernel müssen die gemeinsame Planung überhaupt unterstützen.
Das rechte Bild ist das Labor: M7 ohne RTOS-Kernel, M4 mit FreeRTOS. Links ist die Definition von SMP, nicht der heutige H755-Zustand. Ein gemeinsamer Speicher ersetzt den Kanal nicht.
Zuerst sehen wir, was ein SMP-Scheduler leisten muss.

Folie 7 / 55

Symmetrisches Multiprocessing (SMP)

- Ein Scheduler sieht die Kerne
- Eine Task kann, wenn die Politik das erlaubt, den Kern wechseln
- Dabei werden Register gesichert, nicht der ganze Cache „genullt“
- Harte Fristen sind unter Annahmen möglich und unter anderen Annahmen schwer

Notizen 7 / 55

SMP heißt: ein Scheduler sieht mehrere Kerne. Eine Task kann, wenn die Politik und die Affinität es erlauben, auf einem Kern starten, unterbrochen werden und auf einem anderen weiterlaufen. Migration überträgt den gesicherten Ausführungskontext, also Register und den Verweis auf den Taskstack. Sie setzt den Cache des Zielkerns nicht auf null. Der Cacheinhalt ist ein eigener Speicher. Die Kosten und die Zugriffszeiten können sich ändern. Feste Migrationszeiten werden hier nicht erfunden. Harte Fristen bleiben möglich, wenn die Annahmen und Schranken benannt sind. Sie werden nicht dadurch unmöglich, dass mehr als ein Kern existiert.
Der Titel dieser Folie beschreibt die Möglichkeit. Die nächste Folie benennt die Analyseprobleme, nicht ein Verbot.
Wie verteilen verschiedene SMP-Scheduler die bereiten Tasks?

Folie 8 / 55

Wie funktioniert ein SMP-Scheduler?

- Globale Queue: ein Kern nimmt die höchste bereite Aufgabe
- Oder: Warteschlange je Kern, der Scheduler schiebt nur bei Affinität oder Last
- Beides ist SMP, solange es **ein** Betriebssystemkontext ist
- Keine Pflicht zu genau einer Ready-Liste

Notizen 8 / 55

Eine globale Ready-Queue lässt einen freien Kern die höchste passende Aufgabe nehmen. Kernbezogene Queues halten Aufgaben bei einem Kern. Der Scheduler schiebt nur, wenn Affinität und Last das erlauben. Beides bleibt SMP, solange ein Betriebssystemkontext plant. Eine kernbezogene Queue macht das System nicht zu AMP. Zwei Kerne dürfen nicht dieselbe Taskinstanz gleichzeitig ausführen, nur weil sie bereit ist. „Die höchste Aufgabe“ ist ohne Politik, Affinität und gesperrte Schedulerdaten zu ungenau.
FreeRTOS läuft in diesem Labor nur auf dem M4. Es gibt keine Migration seiner Tasks auf den M7. Diese gemeinsame Planung bringt besondere Echtzeitfragen mit sich.

Folie 9 / 55

Echtzeit-Herausforderungen bei SMP

- Wer darf wohin migrieren?
- Geteilte Caches und Busse erzeugen Interferenz
- Sperren müssen kernübergreifend gedacht werden
- Das ist kein Beweis, dass SMP nie hart echtzeitfähig ist

Notizen 9 / 55

SMP bleibt möglich. Die Analyse wird größer als auf einem Kern. WCET, Worst-Case Execution Time, ist die obere Schranke der Ausführungszeit unter benannten Annahmen. Störquellen sind gemeinsame Speicherpfade, auf manchen Architekturen geteilte Caches, Migration und kernübergreifende Sperren. Der H755 hat keinen gemeinsamen L1-Cache zwischen M7 und M4. Das allgemeine SMP-Bild darf das nicht suggerieren. Eine Auslastung unter der Kernzahl ist keine universelle Deadlinegarantie für jedes Schedulingmodell. Affinität und Partitionierung vereinfachen manche Rechnung und beseitigen die gemeinsamen Ressourcen nicht.
Frage: Kann geeignete Affinität Fristen tragen? Ja, wenn Budgets und ein Interferenzmodell dazukommen. Sie ist kein Verbot von SMP.
AMP macht die Zuständigkeiten expliziter.

Folie 10 / 55

Asymmetrisches Multiprocessing (AMP)

- Zwei Firmware- und oft zwei OS-Kontexte
- Beispiel allgemein: Cortex-A mit Linux und Cortex-M mit RTOS
- Beispiel **dieser** Chip: M7 Bare-Metal und M4 FreeRTOS
- Der H755-M7 ist hier kein Linux-Standard

Notizen 10 / 55

AMP heißt: getrennte Firmware und oft getrennte Betriebssystemkontexte. Das allgemeine Beispiel, Linux auf einem Cortex-A und ein RTOS auf einem Cortex-M, ist nicht der H755. Auf diesem Chip läuft der M7 Bare Metal und der M4 FreeRTOS. Der M7 ist hier kein Linux-Standard. RTOS ist das Echtzeitbetriebssystem, OS das Betriebssystem, Bare Metal das Programm ohne diesen Kernel. Der gemeinsame Chip hat trotzdem gemeinsame Takte, Speicher und Peripherie. Ein Kern kann dem anderen eine Anfrage stellen, ohne dessen Scheduler oder Heap zu benutzen.
Die beiden Beispiele dürfen nicht in eine Zeichnung rutschen.
Die Architektur braucht eine nachvollziehbare Aufteilung der Software.

Folie 11 / 55

Software-Aufteilung in AMP

- M4: Tasks, Tick, Queues **dieses** Kernels
- M7: Superloop oder eigene Struktur, ohne diese Queue-IDs
- Eine lokale `osMessageQueueId_t` im Shared SRAM wird keine Inter-Core-Queue
- Der Kanal ist ein vereinbartes Speicherlayout plus Protokoll

Notizen 11 / 55

Jeder Kern hat eigene Firmware, eigenen Stack, eigenen Heap und, nur der M4, eigene Kernelobjekte. Ein Handle ist ein lokaler Verweis auf eine Struktur, die dieser Kernel verwaltet. `osMessageQueueId_t` ist der Handle einer CMSIS-RTOS2-Queue. SRAM, Static Random Access Memory, ist der Arbeitsspeicher. Liegt nur die Zahl des Handles im gemeinsamen SRAM, fehlen auf dem M7 die Listen und die Wartelogik. Die vorige Vorlesung behandelte Inter-Task-Kommunikation innerhalb eines Kernels. Hier ist Inter-Prozessor-Kommunikation zwischen zwei Firmwarekontexten gemeint. Der M4 kann eine lokal empfangene Nachricht in das vereinbarte IPC-Format schreiben. IPC ist Inter-Processor Communication. Lokale Zeiger und Handles werden nicht ungeprüft übertragen.
Frage: Warum darf die Queue-ID nicht der Kanal zum M7 sein? Weil Handle, Heap und Listen dem M4-Kernel gehören. Der M7 führt diesen Kernel nicht aus.
Welche Kanäle sind dafür geeignet?

Folie 12 / 55

Wie kommunizieren AMP-Kerne?

Mittel	hier
Software-Puffer	unser SRAM4-Block
Hardware-Mailbox / FIFO	anderes Gerät, nicht dieser Ping-Pong
HSEM	Boot-Signal, später auch Sperre
lokale RTOS-Queue	nur M4-intern

Notizen 12 / 55

Shared Memory ist der Speicherort, den beide erreichen. Das Protokoll bestimmt Inhalt, Zustand und wer schreiben darf. Eine Hardware-Mailbox oder FIFO, First In First Out, ist eine gerätespezifische Einrichtung. Aus einer allgemeinen Grafik folgt nicht, dass der H755 sie für diesen Ping-Pong hat. HSEM kann benachrichtigen oder später Zugriffe koordinieren. Die Sperre selbst enthält die Nutzdaten nicht. Die lokale RTOS-Queue bleibt im M4-Kernel. RTOS ist das Echtzeitbetriebssystem, IPC die Vereinbarung zwischen den Firmwares. Heute pollen beide Seiten den Zustand. Das ist eine Lehrvereinfachung, kein Leistungsoptimum.
Datenpfad und Benachrichtigungspfad sind verschieden: SRAM4 trägt die Bytes, HSEM weckt im Boot.

Notizen, Fortsetzung 12 / 55

Die erste Zusammenfassung hält fest, wer plant und was trotzdem stört.

Folie 13 / 55

Zusammenfassung Block 1

- $U \leq 1$ bleibt ein Ein-Kern-Satz mit Annahmen
- SMP teilt einen Scheduler, AMP teilt ihn nicht
- M7-Rechnen ohne gemeinsame Zugriffe stört den M4 weniger als M7-Buslast
- „100 % M7-Last wirkt nie auf den M4“ ist falsch

Notizen 13 / 55

In SMP plant ein Scheduler für mehrere Kerne. In AMP plant jeder Firmwarekontext für sich. Im Labor rechnet der M7 ohne RTOS, der M4 mit FreeRTOS. $U \leq 1$ bleibt der Ein-Kern-Satz mit den Annahmen aus der Agenda. Er gilt nicht für zwei Kerne. Hundert Prozent Last sind nicht gleich: Code und Daten im lokalen TCM des M7, Tightly Coupled Memory, treffen den M4 weniger als wiederholte Zugriffe auf SRAM4. TCM ist eng an den Kern gekoppelter Speicher. Die Voraussetzung muss genannt sein, etwa keine gemeinsamen SRAM-Zugriffe und keine zusätzlichen Interrupts in diesem qualitativen Bild.
Frage: Welche Ressource kann trotz getrennter Firmware beide Kerne verzögern? Eine gemeinsam benutzte Speicherbank, Brücke oder Taktkonfiguration.
Der Hardwareblock zeigt diese gemeinsamen Wege.

Folie 14 / 55

Hardware Deep-Dive: Der STM32H755

Vereinfachte Karte, Details in RM0399 der verwendeten Revision:

Domäne	in dieser Zeichnung
D1	M7, TCM, AXI, Flash-Anbindung
D2	M4, viel AHB-Peripherie, Ethernet 10/100, USB
D3	SRAM4, langsame I/Os

Nicht jede schnelle Peripherie liegt in D1. Ethernet ist nicht Gigabit.

Notizen 14 / 55

D1, D2 und D3 sind Domänenbezeichnungen des STM32H755, keine Tasknamen. TCM ist der eng gekoppelte Speicher, AXI, Advanced eXtensible Interface, und AHB, Advanced High-performance Bus, sind Busfamilien. SRAM4 liegt in D3. Die Tabelle ist eine Auswahl aus RM0399 der verwendeten Revision, keine vollständige Peripheriekarte. „D3 gleich langsame I/Os“ ist zu grob. Für das Labor zählen SRAM4

Notizen, Fortsetzung 14 / 55

und die Low-Power-Funktionen dieser Domäne. Ethernet 10/100 ist kein Gigabit. Nicht jede schnelle Peripherie liegt in D1.
Wer ein Blockbild aus dem Handbuch übernimmt, prüft die Revision. Die Zeichnung hier bleibt bewusst grob.
Die beiden Kerne haben unterschiedliche Speicher- und Ausführungseigenschaften.

Folie 15 / 55

Der Cortex-M7 (Das Muskelpaket)

- Spec bis 480 MHz, L1-I- und D-Cache, TCM
- TCM ist nicht der direkte Arbeitsplatz des M4
- Cache macht wiederholte Zugriffe schnell und fremde Werte unsichtbar
- Labor darf 400 MHz nur nach Clock-Tree-Prüfung sagen

Notizen 15 / 55

Der M7 hat einen L1-I-Cache für Instruktionen und einen L1-D-Cache für Daten. L1 ist die erste Cacheebene. ITCM und DTCM sind Instruction und Data Tightly Coupled Memory, also Befehls- und Datenspeicher eng am Kern. Ein Pfad läuft CPU, Cache oder TCM. Ein anderer läuft CPU, Interconnect, SRAM4. Direkte M4-CPU-Zugriffe auf das M7-TCM sind kein geeigneter gemeinsamer Variablenzugriff. AN5617 unterscheidet das von indirekten Wegen über MDMA, Master Direct Memory Access. Diesen Weg benutzt das Basislabor nicht. Schneller lokaler Zugriff und eine veraltete Datenkopie sind verschiedene Probleme. Die Spec nennt bis 480 MHz. 400 MHz darf das Labor nur sagen, wenn der Clock-Tree das zeigt.
Eine Shared-Variable im TCM wäre für den direkten M4-Zugriff ungeeignet.
Auch der M4 ohne diesen L1-Cache hat zeitliche Einflussfaktoren.

Folie 16 / 55

Der Cortex-M4 (Das Präzisionswerkzeug)

- Spec bis 240 MHz
- Kein M7-artiger L1-Daten-/Instruktionscache
- Der ART-Beschleuniger auf dem Flash-Pfad bleibt
- Wait-States, Bus, IRQs und die Speicherregion bleiben in der Zeit
- „Jeder Zugriff gleich viele Takte“ und „auf die Nanosekunde“ streichen

Notizen 16 / 55

Der M4 in diesem Chip hat keinen L1-Instruktions- und Datencache wie der M7. ART, Adaptive Real-Time Accelerator, beschleunigt den Flash-Pfad des STM32. Die Einzelheiten stehen in der passenden ST-Dokumentation. Wait States sind zusätzliche Wartezyklen beim Speicherzugriff, kein zweiter Scheduler.

Notizen, Fortsetzung 16 / 55

IRQ, Bus und die gewählte Speicherregion ändern die Ausführungszeit. Ohne L1-Cache sind weder alle Instruktionen gleich lang noch die Zeiten auf die Nanosekunde fest. 240 MHz ist das spezifizierte Maximum, nicht die schon gewählte Laborfrequenz.
Ohne L1-D-Cache ist der M4 nicht interferenzfrei. Gemeinsame Busziele treffen ihn trotzdem.
Entscheidend sind auch die Wege vom Kern zum Speicher.

Folie 17 / 55

Die Bus-Matrix (AHB / AXI)

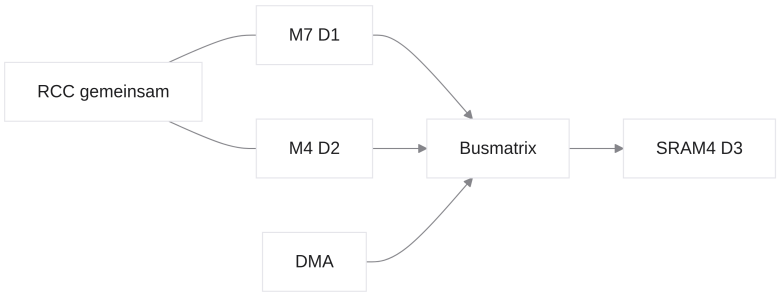
- Parallele Wege, wenn die Pfade verschieden sind
- Dieselbe SRAM-Bank oder dieselbe Brücke serialisiert
- Ein Konflikt braucht nicht denselben CPU-Takt
- „AXI-SRAM immer 1 Takt“ ist keine universelle Garantie

Notizen 17 / 55

AHB und AXI sind die Busfamilien auf diesem Chip. Ein Busmaster startet einen Zugriff, ein Ziel, oft Slave genannt, bedient ihn. Eine Brücke verbindet Busbereiche. Arbitration entscheidet, wer bei einem Konflikt zuerst darf. Die Matrix erlaubt gleichzeitige Zugriffe auf verschiedene freie Ziele. Dieselbe begrenzte SRAM-Bank oder dieselbe Brücke serialisiert. Ein Konflikt braucht nicht denselben CPU-Takt. „AXI-SRAM immer ein Takt“ ist keine universelle Garantie. DMA ist ein weiterer Master und kann denselben Port belegen. Die Pfeile zeigen Konkurrenz, keine gemessene Wartezeit.
Die Matrix beseitigt Interferenz nicht. Sie verschiebt sie auf die gemeinsamen Ziele.
Diese Wege liegen innerhalb und zwischen den drei Domänen.

Folie 18 / 55

Die 3 Power-Domänen (D1, D2, D3)



Oben getrennte Rechenkerne, unten gemeinsame Ziele.

Notizen 18 / 55

Die Zeichnung trennt die Rechenkerne von den gemeinsamen Zielen. M7 sitzt in der Darstellung bei D1,

Notizen, Fortsetzung 18 / 55

M4 bei D2, SRAM4 bei D3. Einen DMA-Block ordnen wir erst einer Domäne zu, wenn die konkrete Instanz im Handbuch geprüft ist. RCC, Reset and Clock Control, konfiguriert Takte. Es ist kein Datenpfad, der alle Nutzbytes transportiert. Power-Domäne, Taktbereich und Zugriffsweg sind drei verschiedene Schnitte. Eine M7-Schleife mit Code im ITCM und Daten im DTCM vermeidet bestimmte SRAM4-Zugriffe, solange keine Interrupts und keine weiteren Master in diesem qualitativen Bild dazukommen. Ob ein Speicher in einem Low-Power-Modus erhalten bleibt, hängt vom Modus ab.
WCET, die obere Laufzeitschranke, wird hier nicht beziffert. Sobald der M7 SRAM4, DMA oder RCC anfasst, kann der M4 warten.
Der Taktbaum wird trotzdem gemeinsam konfiguriert.

Folie 19 / 55

Taktversorgung (RCC)

- Ein Taktbaum, keine private PLL je Kern
- Wer `SystemClock_Config` zweimal ausführt, schreibt dieselben Register
- Im Labor initialisiert der M7 die gemeinsamen Takte, bevor der M4 weiterläuft
- Maxima 480/240 MHz zeichnerisch von der gewählten Laborfrequenz trennen

Notizen 19 / 55

RCC konfiguriert die Takte zentral. Eine PLL, Phase-Locked Loop, erzeugt aus einer Quelle höhere oder stabilere Takte. Mehrere PLLs im Chip sind nicht automatisch private PLLs je Kern. Eine gemeinsame Quelle kann über Teiler verschiedene Kerntakte ergeben. Den konkreten Baum zeichnet Cube, nicht diese Folie. Der M7 setzt die gemeinsamen Takte, während der M4 im vereinbarten Bootzustand wartet. Zwei Aufrufe von `SystemClock_Config` schreiben dieselben Register und können sich widersprechen. 480 und 240 MHz bleiben die Spec-Maxima. Die Laborfrequenz steht im erzeugten Baum.
GPIO-Ports teilen Konfigurationsregister. Das Beispiel dazu gehört zur Pin-Folie. Hier bleibt die Zuständigkeit für den Takt.
Diese Zuständigkeit erklärt den Boot-Handshake.

Folie 20 / 55

Der Boot-Prozess: Wer weckt wen?

Übliches CubeMX-Muster, nicht „M4 bleibt in Reset“:

1. Option Bytes legen fest, wer überhaupt startet; Standard oft: beide Bootcodes
2. M4 richtet eine HSEM-Benachrichtigung ein und geht in **Stop**
3. M7 wartet auf diesen Zustand, setzt Takt und Shared-Block
4. M7 gibt das HSEM frei, M4 läuft weiter ins RTOS

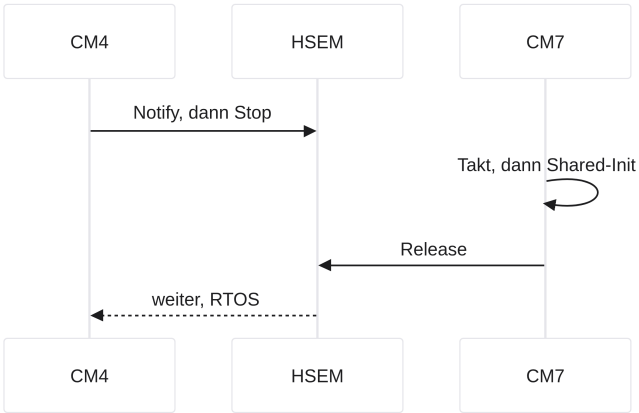
Stop, Reset und abgetakteter Kern sind drei Zustände.
Die konkrete HSEM-ID und die Funktionsnamen stehen in der generierten `main`. `HAL_PWREx_ReleaseCore` nicht aus dem Gedächtnis ergänzen.

Notizen 20 / 55

Option Bytes sind die nichtflüchtige Startkonfiguration und gehören zu den zwei Images. Im üblichen CubeMX-Muster führt der M4 zuerst Bootcode aus, richtet die HSEM-Benachrichtigung ein und geht in Stop. Er bleibt nicht einfach in Reset. Der M7 wartet auf diesen Stopzustand, setzt gemeinsame Takte und den Shared-Block und gibt das HSEM frei. Danach setzt der M4 seine Initialisierung und den RTOS-Start fort. HAL, Hardware Abstraction Layer, liefert die Funktionen des Templates. `HAL_PWREx_ReleaseCore` wird nicht aus dem Gedächtnis ergänzt. Reset heißt noch kein Anwendungscode nach dem Rücksetzen. Stop heißt: Bootcode lief, der Kern wartet auf das Ereignis. Ein abgetakteter Kern ist ein dritter Zustand.
Die Wartezeit des M7 ist begrenzt. Sonst eine Diagnose, keine stumme Endlosschleife. Dieser HSEM-Schritt ist der Boot, nicht die Sperre für jedes Payload-Byte.
Das folgende Bild verbindet die Schritte mit den Prüfstellen.

Folie 21 / 55

Hardware-Register: Den M4 wecken



Warten ohne Fortschritt: Diagnosekasten, nicht stiller Reset.

Notizen 21 / 55

Das Ablaufbild ist eine Reihenfolge, keine Zeitmessung. Der M4 meldet sich und geht in Stop. Der M7 wartet auf diesen Zustand, initialisiert den Block und gibt das HSEM frei. HSEM-Takt, Benachrichtigung und Freigabe der benutzten Semaphore sind getrennte Schritte. Die ID muss auf beiden Seiten zum generierten Template passen. Register- oder HAL-Namen erklären wir an ihrer Funktion im Template, nicht als erfundenen Vollcode. Der M7-Wartepfad hat ein Pollbudget oder eine lokale Zeitbasis. Ein Timeout ist Diagnose. Er erlaubt nicht, den Block neu zu beschreiben, während die Gegenstelle noch in derselben Phase schreibt. AN5361 und das eigene Template gewinnen, wenn die Option Bytes anders stehen.
Vorlage und Projekt dürfen auseinanderlaufen. Dann gilt das Projekt, und die Folie bleibt das Lehrbild des üblichen Musters.
Nach dem Start brauchen auch die Peripherieblöcke feste Besitzer.

Folie 22 / 55

Pin-Zuweisung im Dual-Core

- Pin Context steuert, **welcher Generator** den Init-Code schreibt
- Das ist keine Hardware-Sperre für den anderen Kern
- RCC und jeder Peripherieblock brauchen eine Zuständigkeit
- Gleicher GPIO-Port, verschiedener Pin: immer noch ein Register

Notizen 22 / 55

GPIO, General-Purpose Input/Output, konfiguriert Pins über gemeinsame Register. Zwei Pins desselben Ports teilen zum Beispiel MODER. Pin Context in CubeMX steuert nur, welcher Kern den Code erzeugt. Es sperrt den anderen Kern nicht in Hardware. Lesen beide denselben alten Wert, ändern verschiedene Felder und schreiben zurück, kann der zweite Schreibzugriff die erste Änderung löschen. Das ist Read-Modify-Write auf dem Konfigurationsregister. Atomare Set- und Reset-Register für Ausgangsdaten sind ein anderer Fall und heben dieses Beispiel nicht auf. Ein Besitzer je gemeinsam zu konfigurierendem Peripherieblock nimmt die Änderung entgegen. „Ein Pin je Kern“ genügt nicht, wenn das Register mehrere Pins enthält.
Beide Kerne dürfen `GPIOE->MODER` nicht gleichzeitig lesen, ändern und zurückschreiben.
Die Zuständigkeit muss in den zwei Firmwareprojekten sichtbar sein.

Folie 23 / 55

Toolchain und Projektstruktur

- Zwei ELF-Dateien, zwei Vektortabellen, zwei Linkerlayouts
- CM7 in Bank 1 und CM4 in Bank 2 ist eine mögliche Konvention
- Startadresse und Option Bytes müssen dazu passen
- Ab CubeIDE 2.0 oft: CubeMX separat, dann Import
- Eine Launch-Gruppe ist nicht automatisch da

Notizen 23 / 55

ELF, Executable and Linkable Format, ist das ladbare Image. IDE, Integrated Development Environment, ist hier STM32CubeIDE. GCC, GNU Compiler Collection, übersetzt. Zwei Teilprojekte haben zwei Startup-Dateien, zwei Vektortabellen, zwei Linker-Skripte und einen gemeinsamen Header. Die Vektortabelle enthält lokale Exception- und Interrupt-Einstiege, nicht die IPC-Nachrichten. CM7 in Flashbank 1 und CM4 in Bank 2 ist ein mögliches Layout. Startadressen und Option Bytes müssen dazu passen. AN5361 Rev. 5 beschreibt ab CubeIDE 2.0 den Weg über das eigenständige CubeMX und den anschließenden Import. Ein leeres IDE-Projekt ist ein anderer Weg. Menüscreenshots werden nicht erfunden. Die installierte Version steht am Laborrechner.
Zwei Images brauchen einen abgestimmten Debugstart.

Folie 24 / 55

Debugging im AMP-System

- Eine ST-LINK- oder OpenOCD-Sitzung, nicht beides gemischt beschreiben
- Zwei Images laden, Reset-Reihenfolge zum Boot-Handshake passen
- „Halt all cores“ und Cross-Trigger können den Stop-Weckruf verfälschen
- Nach dem Halt die HSEM- und Stop-Zustände prüfen, nicht nur den PC

Notizen 24 / 55

AMP bleibt die getrennte Firmware. GDB, GNU Debugger, spricht über einen Server mit dem Chip. Im Kurs ist das typischerweise der ST-LINK GDB Server. OpenOCD ist ein anderer Server und wird nicht in denselben Ablauf gemischt, solange er nicht der verwendete ist. Laden beider Images, Reset, Verbindung zum laufenden M4 und das Halten der Kerne sind verschiedene Schritte. AN5361 nennt dafür Einstellungen. Portnummern und Access-Port-Werte sind keine universellen Kurszahlen. „Halt all cores“ kann den M4 hindern, den Stopzustand zu erreichen, auf den der M7 wartet. Cross-Trigger hält weitere Kerne an. Verstrichene Zeit im Halt ist kein Normalbetrieb.
Der Klickpfad hängt von der IDE-Version ab und bleibt offen, solange kein Projekt im Repository liegt.
Die Hardwarezusammenfassung führt zum gemeinsamen Speicherlayout.

Folie 25 / 55

Zusammenfassung Block 2

- D1/D2/D3 grob, Ethernet in D2, 10/100
- Bus parallel nur auf freien Wegen
- Boot: Stop und HSEM, nicht Reset-Leitung
- Zwei Images, eine Shared-Adresse

Notizen 25 / 55

Domänen, Buskonflikt, gemeinsamer Takt und der Boot hängen zusammen. D1, D2 und D3 trennen Rechenorte und Ziele. HSEM gibt im Boot dem M4 das Signal weiterzulaufen. ELF ist je Kern ein eigenes Image. Der M7 wartet auf den Stopzustand. Ein Debughalt aller Kerne kann verhindern, dass der M4 diesen Zustand erreicht. Das ist eine Diagnose, kein Cachefehler. Boot und spätere Nutzdaten sind verschiedene Protokollphasen. Nach erfolgreichem Boot haben zwei gleichnamige C-Variablen noch nicht automatisch dieselbe Adresse.
Als Nächstes stellt der Linker genau diese Übereinstimmung her.

Folie 26 / 55

Die Speicherarchitektur (RAM)

Übliche H755-Anker, im Datenblatt der Revision prüfen:

Region	Adresse	Rolle im Labor
DTCM/ITCM	M7-lokal	nicht der Shared-Puffer
SRAM1–3	D2	M4-Heap/Stack, nicht automatisch shared
SRAM4	`0x38000000`–`0x3800FFFF`	64 KiB, gewählter Kanal
Backup-SRAM	andere Adresse	nicht SRAM4, nicht immer im Standby erhalten

SRAM4 ist die **Laborentscheidung**, keine Vorschrift für jede Anwendung.

Notizen 26 / 55

RAM ist Arbeitsspeicher, SRAM die statische Ausführung auf diesem Chip. ITCM und DTCM gehören zum M7 und sind kein allgemeiner Briefkasten. DMA kann je nach Instanz andere Bänke erreichen. Für den betrachteten Baustein liegt SRAM4 bei `0x38000000` bis einschließlich `0x3800FFFF`, also 64 KiB. Ein KiB sind 1024 Byte. Adresse, Größe, Erreichbarkeit, Cacheattribut und die Linker-Zuordnung sind verschiedene Eigenschaften. SRAM1 bis SRAM3 sind nicht von der Hardware ausschließlich M4-Heap. Das wäre eine Projektentscheidung. Backup-SRAM ist ein anderer Bereich. Ob er in einem Low-Power-Modus erhalten bleibt, hängt vom Modus ab. Bandbreite und DMA können eine andere Bank nahelegen. Das Basislabor bleibt bei SRAM4.

Notizen, Fortsetzung 26 / 55

Beide Projekte reservieren diesen Bereich ausdrücklich für den Lehrkanal.

Folie 27 / 55

Wo legen wir den Shared Memory an?

- Beide Linker reservieren **dieselbe** Spanne und legen sonst nichts hinein
- Heap und Stack bleiben außerhalb
- `NOLOAD` heißt: der Loader schreibt die Sektion nicht
- Es heißt nicht: jedes Startup lässt die Bytes in Ruhe
- `.bss`-Zero und `.data`-Copy beider Images prüfen

Notizen 27 / 55

Jeder Linker platzt ein C-Objekt für sich. Derselbe Name garantiert keine physische Adresse. ELF ist das Image, ABI, Application Binary Interface, die Darstellung der Felder. Der gemeinsame Bereich wird in beiden Images für denselben Zweck reserviert. Dort liegen kein privater Heap, kein Stack und kein weiterer Puffer. NOLOAD kennzeichnet den Abschnitt ohne normale Initialisierungsdaten im Image. Es verhindert nicht jede Zero-Schleife im Startup, nicht jeden Debug-Loader und nicht eine spätere erneute Initialisierung. `.data` wird kopiert, `.bss` genullt, jeweils in der eigenen Firmware. Nach Reset ist der Inhalt kein gültiger Protokollzustand, bis der M7 schreibt. Ein zufälliger Nichtnullwert ist kein Startwert 0 und kein Magic.
Der Linker-Vertrag steht im Skript.

Folie 28 / 55

Das Linker-Script (`.ld`)

Schema, keine abgelesene Map:

- Region SRAM4 einmal, nicht ein zweites `RAM_D3` daneben
- Sektion `.shared` mit `NOLOAD`, `KEEP`, Alignment 32
- `ASSERT` auf Größe und Adresse
- Symbol `sd` in **beiden** Maps gleich, sonst ist es nicht derselbe Block

Notizen 28 / 55

MEMORY nennt Ursprung und Länge einer Region. SECTIONS legt fest, welche Ausgabesektion dort liegt. `.ld` ist die Dateiendung des Linker-Skripts. ELF ist das Ergebnis. Die SRAM4-Region heißt im echten Skript so, wie Cube sie erzeugt hat. RAM_D3 und SRAM4 dürfen nicht zweimal dieselbe Spanne belegen. Das Lehrschema sammelt `.shared` mit KEEP und NOLOAD und prüft die Adresse von `ipc` mit ASSERT. KEEP hält die Eingabesektion, wenn der Linker sonst unreferenzierte Objekte verwirft. Zwei Maps müssen dieselbe `ipc`-Adresse zeigen. Eine Beispieladresse in der Folie ist synthetisch, solange

Notizen, Fortsetzung 28 / 55

keine Map im Repository liegt. Cube erzeugt die `.ld`. Wir fügen die Sektion ein und duplizieren die Speicherliste nicht.
Die nächste Folie zeigt, welche Eingabesektionen gesammelt werden.

Folie 29 / 55

Eine eigene Sektion anlegen

```
.shared (NOLOAD) :
{
  . = ALIGN(32);
  KEEP(*(.shared))
  . = ALIGN(32);
} >RAM_D3
```

Nur wenn `RAM_D3` schon die SRAM4-Region ist. Sonst den vorhandenen Namen nehmen.

Notizen 29 / 55

NOLOAD beschreibt die Ausgabesektion ohne Initialisierungsabbild. ALIGN verschiebt die Platzierung. KEEP schützt die Eingabesektion vor Garbage Collection. `>RAM_D3` wählt die vorhandene Region. `.shared.*` nur, wenn die Objekte wirklich so heißen. `ipc` muss am vereinbarten Anfang liegen. Ein anderes Objekt davor bricht den Vertrag. 32-Byte-Ausrichtung und ein auf 32 Byte gerundeter Abschnitt machen den 12-Byte-Typ nicht 32 Byte groß. Typgröße, Abschnittsreserve und das 64-KiB-MPU-Fenster sind drei Ebenen. Adresse und Größe prüft der Linker-ASSERT. Die 32-Byte-Ausrichtung bleibt sinnvoll, auch wenn die Basisregion nicht cachebar ist, weil eine spätere Wartungsvariante Zeilengrenzen braucht. Der Compiler muss das konkrete Objekt in diese Eingabesektion legen.

Folie 30 / 55

Zuweisung im C-Code (GCC-Attribute)

```
/* shared_data.h: nur Typ und extern */
extern volatile ipc_block_t ipc
    __attribute__((section(".shared")));

/* genau eine .c je ELF */
volatile ipc_block_t ipc
    __attribute__((section(".shared")));
```

Das Attribut allein setzt nicht die Adresse. Linker, Alignment und weitere Objekte entscheiden.

Notizen 30 / 55

GCC übersetzt, ELF ist das Image, ABI die Feldabbildung. Der gemeinsame Header enthält Typ, Konstanten und `extern volatile ipc_block_t ipc`. Genau eine `.c` je Image definiert das Objekt. Ein gemeinsames Quellmodul darf für beide Images gebaut werden, aber je Image nur einmal. Das ist keine doppelte Definition innerhalb eines Images. `section(".shared")` wählt die Eingabesektion, `aligned(32)` richtet das Objekt aus und ändert nicht das natürliche Alignment des Typnamens, `used` hält die Referenz. KEEP im Linker fängt den Fall ab, dass der Linker das Objekt sonst verwirft. Der Header legt keinen Speicher an und setzt keine Adresse. Die Definition nicht zusätzlich im Header ablegen.
Die Zugriffe brauchen weitere Eigenschaften als nur die Sektion.

Folie 31 / 55

Das Schlüsselwort volatile

- `volatile` zwingt den Compiler zu den Zugriffen, die **dieser** Compiler dafür definiert
- Es umgeht nicht den D-Cache
- Es ist kein Mutex und kein atomares Read-Modify-Write
- Es veröffentlicht nicht „den ganzen Struct auf einmal“

Notizen 31 / 55

`volatile` sagt dem Compiler, dass sich der Wert außerhalb des sichtbaren Ablaufs ändern kann. Eine Pollschleife auf `ipc.state` darf den Vergleich nicht aus dem Register wiederholen, ohne neu zu laden. GCC beschreibt das in der Volatile-Dokumentation. Es verspricht nicht, dass jeder Ausdruck genau einen Buszugriff erzeugt. Der Zugriff kann aus dem D-Cache kommen. `volatile` ist kein Mutex. `ipc.payload += 1` ist keine allgemeine atomare Mehrkernoperation und veröffentlicht nicht alle Felder zugleich. Im Lehrprotokoll ist die Statusvariable ein ausgerichtetes `uint32_t` unter den Annahmen von Armv7-M, Speicherattribut und Toolchain. Das ist keine portable C-Thread-Garantie. Für die Reihenfolge kommt eine Barriere dazu, für den Cache die MPU oder eine geprüfte Wartung.
Die vier Probleme brauchen verschiedene Mittel.

Folie 32 / 55

Reicht volatile für Multicore?

Vier verschiedene Probleme:

Problem	Mittel	löst allein nicht
Compiler hält einen Wert	`volatile`	Cache, Reihenfolge, Konflikt
veraltete Cachezeile	MPU oder Clean/Invalidate	wer schreiben darf
Reihenfolge der Zugriffe	`DMB` / `DSB`	Cache-Inhalt
beide schreiben	Ownership oder HSEM	Sichtbarkeit

`DMB` ordnet. `DSB` wartet auf den Abschluss. Keines reinigt den D-Cache.
Keine Armv8-M-Acquire-Intrinsics ungeprüft auf M4/M7.

Notizen 32 / 55

Vier Fehler, vier Mittel. Der Compiler hält einen Wert: `volatile` zwingt zum erneuten Zugriff und löst Cache, Reihenfolge und Konflikt nicht. Eine veraltete Cachezeile braucht nicht cachebare Attribute oder Clean und Invalidate und sagt nicht, wer schreiben darf. DMB, Data Memory Barrier, ordnet Speicherzugriffe. DSB, Data Synchronization Barrier, wartet auf den Abschluss relevanter früherer Operationen. Keines schreibt alle Dirty-Zeilen zurück. Ownership legt fest, wer das Datum in dieser Phase schreiben darf. Eine Sperre koordiniert und ersetzt die übrigen Punkte nicht. `__ISB`, Instruction Synchronization Barrier, gehört zur MPU-Aktivierung, nicht zum Payload. Acquire-Intrinsics von Armv8-M setzen wir auf M4 und M7 nicht ungeprüft ein.
Frage: Warum reicht nicht cachebar nicht? Weil die Reihenfolge „Payload, dann Zustand“ und das Schreibrecht bleiben. DMB bleibt nötig.
Das Protokoll beginnt mit einem eindeutigen Layout.

Folie 33 / 55

Struktur der Ping-Pong Daten

Altes Bild mit zwei `uint8_t` und einem `uint32_t`: natürliche Ausrichtung **8 Byte**, Offsets 0, 1 und 4, nicht 12.
Gewählter Block:

```
typedef struct {
    uint32_t magic;
    uint32_t state;    /* TURN_M7, REQUEST_M4, RESPONSE_M7 */
    uint32_t payload;
} ipc_block_t;
```

`_Static_assert` auf Größe und Alignment im Zielbuild. Kein `packed` als Multicore-Trick.
Nur der Berechtigte schreibt `payload`. Der Zustand wechselt danach.

Notizen 33 / 55

`uint32_t` ist eine vorzeichenlose 32-Bit-Zahl. ABI ist die Abbildung im Zielcompiler. Im gewählten Layout liegen magic bei Offset 0, state bei 4 und payload bei 8. `sizeof` ist 12, `offsetof` prüft die Offsets, `_Alignof` mindestens 4. Eine ältere Variante mit zwei `uint8_t` und einem `uint32_t` braucht im üblichen Ziel-ABI oft 8 Byte mit Padding, nicht automatisch 12. Das ist kein Satz für jede ABI. `packed` ist kein Synchronisationsmittel. `TURN_M7` ist der Initialisierungszustand, bevor der M4 kommuniziert. Danach wechseln `REQUEST_M4` und `RESPONSE_M7`. Nur der Berechtigte der Phase schreibt die Payload. Der Zustand wechselt danach, von beiden Kernen abwechselnd.
Der M7 schreibt Magic, Zustand und Payload, solange der M4 noch nicht kommuniziert. NOLOAD-Inhalt ist kein Startwert 0.
Erst Layout plus Initialisierung machen den Block benutzbar.

Folie 34 / 55

Zusammenfassung: Shared Memory

- Eine Adresse, zwei ELF, eine Definition je Image
- M7 initialisiert, M4 löscht den Block beim Start nicht
- Ein Core-Reset mitten im Ping-Pong ist nicht das Basislabor
- Ein gemeinsamer Reset muss den Handshake neu durchlaufen

Notizen 34 / 55

Der Vertrag ist: dieselbe physische Adresse, derselbe Typ und Konstantensatz, eine Definition je Image, keine fremde Belegung der Spanne, explizite Initialisierung durch den M7. ELF ist je Kern ein Image, IPC der Kanal. Beide Programme haben getrennte Startup-Pfade und sprechen dieselben Bytes an. Der M4

Notizen, Fortsetzung 34 / 55

darf die Region später nicht wieder nullen. Ein unabhängiger Reset eines Kerns mitten im Dialog braucht ein Wiederanlaufprotokoll und ist aus dem Basislabor ausgeschlossen. Ein gemeinsamer Reset durchläuft die Initialisierung erneut.
Frage: Was steht nach dem Reset vor der Init? Undefiniert, bis der M7 schreibt. Ein passender Magic-Wert danach ist die Plausibilität des koordinierten Boots, keine Heilung eines einseitigen Resets.
Ein korrektes Layout garantiert noch nicht, dass beide dieselben aktuellen Werte lesen.

Folie 35 / 55

Was ist ein CPU-Cache?

- Zeile typisch 32 Byte auf diesem M7
- Ein Lese-Treffer kann einen alten Wert liefern
- Ein Schreib-Treffer kann nur die Zeile ändern
- Der M4 sieht SRAM, nicht die Dirty-Line des M7

Notizen 35 / 55

CPU ist der Kern, SRAM der Speicher, den der M4 direkt sieht. Eine Cachezeile des M7 ist hier typisch 32 Byte und kann mehrere Variablen enthalten. Ein Hit liefert Daten aus der Kopie und kann schneller und trotzdem veraltet sein. Ein Miss holt aus dem Speicher. Valid heißt, die Kopie darf benutzt werden. Dirty heißt, sie enthält Schreibdaten, die der SRAM noch nicht hat. Write-Back ändert zuerst die Zeile. Der M4 liest den SRAM, nicht diese Dirty-Line. Der 12-Byte-Block liegt in einer 32-Byte-Zeile. Nachbarbytes werden mitgezogen.
Auch Lesen ist ein Kohärenzproblem. Eine alte gültige Kopie verdeckt neuere SRAM-Daten.
Die Schreibpolitik bestimmt, wann geänderte Kopien zurückgehen.

Folie 36 / 55

Write-Through vs. Write-Back

- Write-Through: Schreiben geht Richtung Speicher, hilft nicht für die Richtung M4 → M7
- Write-Back: Rückschreiben bei Verdrängung oder Clean, nicht nur „wenn der Cache voll ist“
- Clean schreibt Dirty-Daten
- Invalidate verwirft die **Kopie**, nicht benachbarte SRAM-Bytes
- Verloren gehen Dirty-Daten, die noch nicht zurückgeschrieben waren

Notizen 36 / 55

Write-Through schickt Schreibdaten in Richtung Speicher. Das macht eine neue M7-Schreibung eher im SRAM sichtbar, aktualisiert aber keine alte M7-Lesekopie, nachdem der M4 geschrieben hat. Write-Back

Notizen, Fortsetzung 36 / 55

schreibt Dirty-Daten bei Verdrängung oder bei einem expliziten Clean zurück, nicht nur „wenn der Cache voll ist“. Clean behält eine gültige Kopie und schreibt Änderungen. Invalidate verwirft die Kopie. Schon im SRAM stehende Nachbarbytes werden dadurch nicht gelöscht. Dirty-Bytes, die noch nicht zurück sind, können verloren gehen. Eine alte Dirty-Zeile über neuere M4-Daten zu cleanen überschreibt die Antwort. Deshalb kein pauschales Clean und Invalidate des gemischten 12-Byte-Blocks.
Aus diesen Eigenschaften entstehen zwei getrennte Kohärenzfehler.

Folie 37 / 55

Das Drama der Cache-Kohärenz

Zwei Zustände, dieselbe Zeile:

1. M7 hat den neuen Request nur dirty. M4 liest den alten SRAM-Wert.
2. M4 hat die Antwort ins SRAM geschrieben. M7 liest noch seine alte Kopie.

Notizen 37 / 55

Fall A: der M7 schreibt eine neue Anfrage in eine Write-Back-Zeile. Der SRAM bleibt zunächst alt. Der M4 kann die neue Anfrage nicht zuverlässig sehen. Fall B: der M4 hat die Antwort in den SRAM geschrieben. Der M7 hat noch eine gültige alte Kopie und liest daraus. Beide Fälle betreffen Payload und Zustand, wenn beide cachebar liegen. Protokollzustände wie REQUEST_M4 und Cachezustände wie dirty sind verschiedene Dinge. Die Beispielwerte sind synthetisch und nicht die Laborfolge 1, 3, 5. Es gibt hier keine maßstabsgetreue Zeitachse, weil keine Latenz gemessen wird. Die logische Reihenfolge der zwei Fehler bleibt gültig.
Die nächste Folie stellt die widersprüchlichen Werte direkt nebeneinander.

Folie 38 / 55

Visualisierung: Cache Coherency Failure

Zwei Richtungen, synthetische Werte, keine Laborsequenz. Kein Clean ist eingezeichnet.

Ort	Fall A: M7 schreibt neu	Fall B: M4 schreibt Antwort
M7-D-Cache	payload = 1, dirty	payload = 0, valid
Shared-SRAM	payload = 0	payload = 1
wer liest	M4 liest 0 aus SRAM	M7 liest 0 aus seiner Kopie

Fall A: die neue 1 liegt nur in der Dirty-Zeile, der SRAM bleibt 0. Fall B: der SRAM hat schon 1, die alte gültige M7-Zeile bleibt 0. Die gestrichelte Kante bedeutet: diese Übertragung findet nicht statt.

Notizen 38 / 55

Die Tabelle hat zwei Spalten und vier Orte: M7-CPU über den D-Cache, den Shared-SRAM und den M4. Fall A: Cache-Payload 1 und dirty, SRAM-Payload 0, der M4 liest 0. Die neue 1 wurde nicht zurückgeschrieben. Fall B: der M4 hat SRAM-Payload 1 geschrieben, die M7-Zeile bleibt 0 und valid, der M7 liest 0. Die gestrichelte Bedeutung ist: diese Übertragung findet nicht statt. Es ist kein Clean eingezeichnet. Die Zahlen illustrieren den Mechanismus und sind nicht die Ping-Pong-Folge. Ein Cache-Hit kann deshalb der falsche Wert sein.
Manche größeren Systeme haben dafür zusätzliche Kohärenzhardware.

Folie 39 / 55

HW-Lösung in PCs: Cache-Snooping

- Kohärenz kann Cache-zu-Cache liefern, nicht nur über RAM
- MESI ist ein Lehrbild, nicht das vollständige Modell jedes PCs
- Daraus folgt nicht, dass PC-Kerne nie hart echtzeitfähig sind
- Der H755 hat diese Kohärenz zwischen M7-L1 und M4 nicht

Notizen 39 / 55

HW heißt Hardware, PC Personal Computer. MESI steht für Modified, Exclusive, Shared und Invalid und ist ein vereinfachtes Zustandsmodell kohärenter Caches. Kohärenz kann aktuelle Daten von einer anderen Cacheinstanz liefern. Das RAM muss nicht nach jedem Schreiben sofort aktuell sein. Das Bild ist kein vollständiges Modell aller PCs und kein Echtzeitnachweis. Der STM32H755 hat keine solche Kohärenz zwischen dem M7-L1 und den M4-Zugriffen. Aus MESI folgen keine M7-Registereinstellungen.
Ohne diese Hardware müssen Software oder Speicherattribute die Kopien behandeln.

Folie 40 / 55

Software-Lösung 1: Manuelles Management

Option, nicht die Basis:

- Getrennte 32-Byte-Blöcke, ein Schreiber
- Clean nur der eigenen neuen Daten, bevor der andere lesen muss
- Invalidate nur, bevor man fremde neue Daten lesen will, und ohne dirty Nachbarschaft zu opfern

Eine Line mit Payload, Flag und fremder Dirty-Variable ist der Fehlerfall.

Notizen 40 / 55

Die Cachewartung ist eine Vertiefung, nicht die Basisimplementierung. CMSIS liefert die D-Cache-Funktionen des M7. SRAM ist der Zielspeicher. Getrennte 32-Byte-Bereiche brauchen je einen Schreiber. Clean betrifft eigene neue Daten vor der Übergabe. Invalidate betrifft eine alte lokale Kopie vor dem

Notizen, Fortsetzung 40 / 55

Lesen fremder Daten. Kontrollinformationen brauchen dasselbe Modell. Getrennte Payloadzeilen lösen ein cachebares Statusflag nicht. Start, Ende und Nachbarobjekte müssen in den Zeilen liegen, die man wartet. Alignment der Startadresse genügt nicht. Ein pauschales Clean und Invalidate des gemischten 12-Byte-Blocks ist keine korrekte Lösung. Die Signaturen stehen in den eingebundenen Headern, nicht nur in der neuesten Onlinedoku.
Warum diese Option im Einsteigerlabor fehleranfällig ist, zeigt die nächste Folie.

Folie 41 / 55

Nachteile der Software-Lösung

- Leicht die Richtung zu verwechseln
- Clean der veralteten Line kann M4-Daten überschreiben
- Invalidate kann eigene Dirty-Bytes verwerfen
- Nur aufnehmen, wenn der Code wirklich geprüft ist
- Das Basislabor macht die Region non-cacheable

Notizen 41 / 55

Zwei Schäden: Eine Zeile, die noch Dirty-Bytes enthält, wird invalidiert, und eigene Änderungen gehen verloren. Eine alte Dirty-Zeile des M7 wird nach einer M4-Antwort gereinigt und überschreibt neuere SRAM-Daten. Dafür muss die Zeile dirty gewesen sein. Clean einer unveränderten sauberen Kopie ist nicht dasselbe Rückschreiben. Cachewartung ist brauchbar, wenn Besitz und die vollständige Sequenz stimmen. Attribute während laufender Kommunikation umzuschalten ist kein Laborumschalter. Zwischen Basis und Fehlervariante gibt es einen gemeinsamen Neustart.
Die Basisvariante vermeidet Cachekopien für den kleinen Block.

Folie 42 / 55

Hardware-Lösung 2 (Best Practice): Die MPU

Für SRAM4 im Basislabor, Attribute aus AN4838, Konstanten im HAL-Header prüfen:

Feld	Basisregion	Fehlerexperiment, eigener Build
Basis / Größe	`0x38000000`, 64 KiB, SIZE-Code 15	dieselbe Spanne, nur zum Vergleich
Typ	Normal, TEX = 001, C = 0, B = 0, S = 1	TEX = 001, C = 1, B = 1, S = 0
Rechte	Lesen/Schreiben, Execute Never	gleich, aber cachebares Write-Back

S = 1 erzeugt kein Snooping. Ein nur geändertes C bei bleibendem S = 1 ist auf dem M7 kein sicherer Nachweis der Cachewirkung. Die Fehlervariante setzt S = 0 und startet gemeinsam neu.
Reihenfolge: MPU, dann D-Cache an, dann Shared-Zugriff.
Der Pfeil umgeht den D-Cache. Ownership und `DMB` bleiben.

Notizen 42 / 55

Der Titel meint das Basislabor, nicht jede Bandbreitenanforderung. MPU ist die Speicherschutzeinheit des M7. TEX, Type Extension, ist das Attributfeld, kein Datenpuffer. XN, Execute Never, verbietet das Ausführen von Code aus der Region. AP, Access Permission, sind die Rechte. C, B und S stehen für cacheable, bufferable und shareable. Die Basisregion ist `0x38000000`, 64 KiB, Normal, TEX 001, C 0, B 0, S 1, Lesen und Schreiben, XN. S gleich 1 erzeugt kein Snooping zwischen M7 und M4. Die MPU ist kernlokal und schützt den M4 und DMA nicht automatisch. Bei Überlappung gewinnt auf Armv7-M die passende Region mit der höheren Nummer. Eine cachebare Region darüber würde die Absicht aufheben. Die Bits stehen im HAL-Header und werden im MPU-Register geprüft.
Welche Probleme diese Region löst und welche bleiben, zeigt die nächste Folie.

Folie 43 / 55

Was bewirkt die MPU?

- Diese Region wird für den M7 nicht als cachebarer Normal-Speicher behandelt
- Sie macht die Bytes nicht „100 % sicher“
- Zwei Kerne ohne Cache brauchen das Protokoll trotzdem
- Weglassen der MPU erzeugt nur dann den Cachefehler, wenn der D-Cache an ist und der Weg cachebar bleibt

Notizen 43 / 55

Für die Shared-Region benutzt der M7 keine entsprechende D-Cachekopie. Andere Regionen dürfen weiter vom Cache profitieren. CCR, Configuration and Control Register, zeigt, ob der D-Cache an ist. DMB bleibt für die Reihenfolge. Ownership bleibt für das Schreibrecht. „MPU aus“ zeigt den Cachefehler nur, wenn der D-Cache an ist und die effektiven Attribute der benutzten Adresse cachebar sind. Ein einmal erfolgreicher Lauf ohne MPU beweist die Korrektheit nicht. Verdrängung und der Debugger können den Fehler verdecken. Die Write-Back-Variante ist ein eigener Build mit Neustart, nicht ein Umschalten im Dialog.
Mit dieser Sichtbarkeit lässt sich das Protokoll vollständig erklären.

Folie 44 / 55

Laborvorbereitung: Der Ping-Pong Datenaustausch

Zustand: `TURN_M7`, `REQUEST_M4`, `RESPONSE_M7`.
M7 setzt vor der Freigabe Payload 0 und den ersten Request.
M4 macht daraus 1. M7 druckt 1, legt 2 hin, requested wieder.
M4 antwortet 3. Sichtbar: **1, 3, 5, …** nicht zuerst 0.
Nach der Übergabe schreibt niemand weiter am Payload.

Phase	Payload danach	Zustand danach	M7 zeigt
M7 init	0	REQUEST_M4	noch nichts
M4 auf 0	1	RESPONSE_M7	noch nichts
M7 liest 1, legt 2	2	REQUEST_M4	1
M4 auf 2	3	RESPONSE_M7	noch nichts
M7 liest 3, legt 4	4	REQUEST_M4	3
M4 auf 4	5	RESPONSE_M7	noch nichts
M7 liest 5, legt 6	6	REQUEST_M4	5

Logische Reihenfolge, keine Zeitachse. Vor jedem Zustandswechsel steht ein DMB nach dem Payload-Schreiben. Nach dem beobachteten Zustand steht ein DMB vor dem Payload-Lesen.

Notizen 44 / 55

`TURN_M7` ist der Initialisierungszustand, bevor der M4 kommuniziert. Danach wechseln `REQUEST_M4` und `RESPONSE_M7`. Der M7 schreibt vor dem Wecken Payload 0 und veröffentlicht den Request. Der M4 liest 0, schreibt 1 und veröffentlicht die Antwort. Der M7 liest 1 in eine private Variable, gibt 1 aus, schreibt 2 und veröffentlicht den nächsten Request. Danach entstehen 3 und 5. Die Tabelle ist die logische Reihenfolge, keine Uhr. IPC ist der Kanal, UART die Ausgabe des M7. Daten werden vor dem Zustandswechsel geschrieben. Nach der Übergabe schreibt dieser Kern die Payload nicht weiter.
Die 500-ms-Pause und die UART-Zeit sind die sichtbare Rate, nicht die IPC-Latenz. `osDelay(1)` ist ein Kernel-Tick, nicht automatisch 1 ms.

Notizen, Fortsetzung 44 / 55

Zuerst werden beide Firmwarekontexte passend erzeugt.

Folie 45 / 55

Schritt 1: CubeMX Dual-Core Setup

- Board nur übernehmen, wenn es NUCLEO-H755ZI-Q ist
- Beide Kerne, FreeRTOS nur auf dem M4, CMSIS_V2
- UART-Retargeting einem Kern zuordnen
- Takt aus dem Cube-Tree abschreiben, nicht aus dieser Folie

Notizen 45 / 55

RTOS und CMSIS-RTOS2 liegen nur auf dem M4. UART, die serielle Schnittstelle, hat einen Besitzer, der `printf` wirklich bedient. VOS, die Spannungsskalierung, und der Clock-Tree entscheiden, welche Kerntakte legal sind. Das steht im erzeugten Projekt, nicht als Behauptung 400 oder 480 MHz auf dieser Folie. CMSIS_V2 ist die Generatorbezeichnung für die RTOS2-Schnittstelle. NUCLEO-H755ZI-Q nur, wenn Cube dieses Board erzeugt hat. Die Übersicht ersetzt keine Screenshots.
Stromversorgung und VOS bleiben offen ohne Projekt.
Die erzeugte Bootsequenz wird bewahrt und um die geordnete Shared-Initialisierung ergänzt.

Folie 46 / 55

Schritt 2: Den Cortex-M4 wecken

Den generierten HSEM/Stop-Code behalten und prüfen:

- HSEM-Takt an
- ID notieren
- M4: Notify, Stop, begrenztes Warten auf M7-Seite
- M7: Shared-Init **danach** erst Release
- Fehler: Status ausgeben, nicht den Block löschen, während der andere läuft

Notizen 46 / 55

Der generierte HSEM- und Stop-Code bleibt. HSEM ist die Hardwaresemaphore, HAL die Abstraktionsschicht, IPC der Datenblock. Der HSEM-Takt ist an, die ID wird notiert und muss auf beiden Seiten gleich sein. Der M4 benachrichtigt und geht in Stop. Der M7 wartet begrenzt, initialisiert den Shared-Block und gibt erst dann frei. Die MPU muss vor dem ersten Shared-Zugriff und in der gewählten Folge vor dem D-Cache wirksam sein, auch wenn die MPU-Folie später kommt. Späterer M4-Startup darf die Region nicht löschen. Eine zweite Wake-Funktion neben dem Template wird nicht erfunden. Ein Timeout meldet den Status und beschreibt den Block nicht neu, während die Gegenstelle läuft.
Dafür brauchen beide Images denselben Datenvertrag.

Folie 47 / 55

Schritt 3: Shared Memory Struct anlegen

Header gemeinsam, Definition je eine `.c` pro Image.
`_Static_assert` im Zielbuild: `sizeof(ipc_block_t) == 12`, Offsets 0, 4 und 8. `sizeof % 4 == 0` beweist das Layout nicht.
`IPC_MAGIC` ist die Konstante `0x49504336`. Der M4 vergleicht `ipc.magic` mit genau diesem Wert. „Ungleich 0“ erkennt keinen alten oder zufälligen Speicher. Ein passender Magic-Wert kann nach einem unabhängigen Core-Reset veraltet sein. Das Basislabor schließt diesen Reset aus und verlässt sich auf den koordinierten Boot.

Notizen 47 / 55

Der Header ist die einzige Quelle für Typ, `IPC_MAGIC` gleich `0x49504336` und die Zustände. ABI und ELF bestimmen, ob die Asserts im Zielbuild gelten: Größe 12, Offsets 0, 4 und 8. `sizeof % 4` beweist das nicht. Der M4 vergleicht Magic mit der Konstanten. Ungleich null genügt nicht, weil alter Speicher beliebige Werte haben kann. Ein passender Magic-Wert kann nach einem unabhängigen Core-Reset veraltet sein. Das Basislabor schließt diesen Reset aus. Der koordinierte Boot ist die Frischeannahme. Der M4 führt kein `memset` auf den Block aus. Die Prüfung sagt etwas über den Wert, nicht über die Aktualität nach einem einseitigen Neustart.
Der Linker muss den Vertrag an derselben Adresse erfüllen.

Folie 48 / 55

Schritt 4: Das Linker-Script anpassen

Beide Skripte: dieselbe Sektion, kein doppeltes RAM.
Map: Symbol `ipc` auf beiden Seiten dieselbe Adresse. Nicht `sd`.
Startup: der Bereich darf nicht in der Zero-Tabelle liegen, sonst löscht ein Kern den laufenden Dialog.

Notizen 48 / 55

ELF ist je Image das Ergebnis. Beide Skripte reservieren dieselbe SRAM4-Spanne und legen `ipc` an dieselbe Adresse. Ein Map-Auszug in der Folie ist synthetisch, solange keine echten Maps vorliegen. Größe, Alignment und Abschnittsgrenzen prüft ASSERT. Im Basislabor bleibt die 64-KiB-Region exklusiv für Shared-Nutzung, weil die MPU sie ganz erfasst. `.bss`-Zero und `.data`-Copy beider Startups dürfen die Sektion nicht enthalten. NOLOAD beschreibt das Ladebild und ist kein Beweis gegen Debugger oder Startup. Die erzeugten ELF-Dateien und der Loader werden geprüft, sobald das Projekt existiert. Nach Adress- und Initialisierungsprüfung verarbeitet der M4 gültige Requests.

Folie 49 / 55

Schritt 5: Die Ping-Pong Logik (Cortex-M4)

```
#define IPC_MAGIC      UINT32_C(0x49504336)
#define IPC_WAIT_TICKS 2000u /* Lehrbudget, Tickfrequenz prüfen *

void IpcTask(void *argument) {
    uint32_t t0 = osKernelGetTickCount();
    (void)argument;
    for (;;) {
        uint32_t waited = osKernelGetTickCount() - t0;
        if (ipc.magic != IPC_MAGIC || ipc.state != REQUEST_M4) {
            if (waited > IPC_WAIT_TICKS) { ipc_error("m4-wait");
            osDelay(1); /* ein Kernel-Tick, nicht zwingend 1 ms
            continue;
        }
        t0 = osKernelGetTickCount();
        __DMB();
        uint32_t v = ipc.payload;
        ipc.payload = v + 1u;
        __DMB();
        ipc.state = RESPONSE_M7;
        /* bis zum nächsten REQUEST_M4 die Payload nicht anfasse
    }
}
```

`ipc_error` ist Lehrdiagnose: Zustand merken, Kanal beenden, Shared-Felder nicht neu beschreiben. `IPC_WAIT_TICKS` ist ein Lehrbudget, keine gemessene Deadline. Die 500-ms-Anzeige des M7 muss in dieses Budget passen.

Notizen 49 / 55

Die Task wartet auf `IPC_MAGIC` und auf `REQUEST_M4`. IPC ist der Block, RTOS die Zeitbasis, DMB die Ordnungsbarriere. `osKernelGetTickCount` liefert Kernel-Ticks, wenn der Kernel läuft. Die Differenz ist unsigned und übersteht einen Zählerumlauf, solange das Budget kleiner als die halbe Zählerweite bleibt und regelmäßig geprüft wird. `IPC_WAIT_TICKS` ist ein Lehrbudget, keine gemessene Deadline. Es muss die bewusste Anzeigepause des M7 enthalten. `osDelay(1)` blockiert diese Task um einen Tick. Andere bereite Tasks und Interrupts laufen. Der M4 steht nicht. Ein Zähler fertiger Runden erkennt eine ausbleibende Antwort nicht, weil dann keine Runde mehr abgeschlossen wird. Nach dem DMB liest die Task die Payload einmal, schreibt den Nachfolger und veröffentlicht `RESPONSE_M7`. Danach fasst sie die Payload nicht an, bis der nächste Request kommt. `ipc_error` beendet den Kanal und beschreibt die Shared-Felder nicht neu.
Die Gegenstelle wartet ebenso strikt auf ihre Schreibberechtigung.

Folie 50 / 55

Schritt 6: Die Ping-Pong Logik (Cortex-M7)

Einmalig im Boot, bevor der M4 kommuniziert: Payload 0, Magic, DMB, `REQUEST_M4`, dann HSEM-Freigabe. Die Laufschleife wartet wirklich auf die Antwort:

```
static int wait_response(uint32_t *v) {
    uint32_t t0 = HAL_GetTick(); /* lokale M7-Zeit, Tick muss la
    for (;;) {
        if (ipc.magic != IPC_MAGIC) return -1;
        if (ipc.state == RESPONSE_M7) {
            __DMB();
            *v = ipc.payload; /* genau einmal, private Kopie */
            return 0;
        }
        if ((HAL_GetTick() - t0) > 2000u) return -1; /* Lehr-ms,
    }
}

uint32_t v;
if (wait_response(&v) != 0) { ipc_error("m7-wait"); return; }
log_u32(v);           /* erste Ausgabe 1, dann 3, dann 5 */
ipc.payload = v + 1u;
__DMB();
ipc.state = REQUEST_M4;
```

Vor `RESPONSE_M7` liest und überschreibt diese Schleife die Payload nicht. `log_u32` und `ipc_error` sind Lehrfunktionen des M7, keine ST-API. `HAL_Delay(500)` darf danach als sichtbare Pause stehen, solange der M4 genug Wartebudget hat. Die Pause ist keine IPC-Latenz.

Notizen 50 / 55

Die Boot-Initialisierung setzt Payload 0 und den ersten Request genau einmal. Die Laufschleife wartet in `wait_response` auf `RESPONSE_M7`. IPC ist der Block, DMB die Barriere, HAL die lokale Tickquelle, UART die Ausgabe. `HAL_GetTick` ist nur eine Zeit, wenn der HAL-Tick in der Warteschleife fortschreitet. 2000 ist eine Lehrgrenze in diesen Ticks, keine Latenzmessung. Vor der Antwort liest und überschreibt die Schleife die Payload nicht. Nach dem Zustand kommt der DMB, dann genau eine private Kopie. `log_u32` gibt sie aus. Die erste Ausgabe ist 1. Danach schreibt der M7 den Nachfolger und veröffentlicht `REQUEST_M4`. `HAL_Delay(500)` darf als sichtbare Pause folgen und ist Busy-Wait auf dem M7, sofern die Implementierung nicht ersetzt wurde. Der M4 kann im RTOS weiterlaufen. `ipc_error` meldet und beendet die Kanalbearbeitung, ohne den fremden Besitz zu überschreiben. Diese Logik setzt die M7-Speicherattribute der nächsten Folie voraus.

Folie 51 / 55

Schritt 7: MPU im M7 konfigurieren

Vor `SCB_EnableDCache()` und vor dem ersten Shared-Zugriff.
Lehrregion: Basis `0x38000000`, 64 KiB, ausgerichtet, Normal, TEX 001, C 0, B 0, Execute Never, Schreiben erlaubt, Shareability passend zu AN4838. Höhere Regionsnummer gewinnt bei Überlappung. Konstantennamen im HAL-Header können abweichen; die Bitwerte sind die Lehrkonfiguration.
Versuch 2, eigener Build, Neustart: Region cachebar, Write-Back, keine Wartung. Fehler diagnostizieren, nicht als garantierten Freeze verkaufen.

Notizen 51 / 55

Die MPU steht vor `SCB_EnableDCache` und vor dem ersten Shared-Zugriff. Die Foliennummer ist nicht die Ausführungsreihenfolge. DSB und ISB gehören zur Aktivierung, wie HAL oder CMSIS sie ausführen. Die Basis ist auf 64 KiB ausgerichtet. Die Größenkodierung von Armv7-M ist $2^{\text{SIZE}+1}$ Byte, also SIZE 15 für 64 KiB. Die HAL-Konstante trägt diese Kodierung, nicht die Bytezahl 65536 als Registerfeld. Regionsnummer und Überlappung werden am vorhandenen Setup geprüft. Eine freie Nummer wird nicht erfunden. Das Hintergrundrecht, etwa privilegierte Default-Zugriffe, muss zum übrigen Programm passen. Die Fehlervariante ist ein eigener Build mit aktivem D-Cache und TEX 001, C 1, B 1, S 0. Ohne aktiven D-Cache zeigt sie nichts. CCR lesen.
Danach können beide Images gemeinsam gestartet und geprüft werden.

Folie 52 / 55

Flashen & Debuggen im Labor

- Beide ELF laden, Reset so, dass der HSEM-Handshake durchläuft
- Erste drei Runden: M4 schreibt 1, 3, 5; M7 druckt sie
- UART-Kern ist der, der `printf` wirklich bedient
- Breakpoints auf beiden Kernen verschieben Timeouts

Notizen 52 / 55

ELF sind die zwei Images, UART die Ausgabe, GDB der Debugger über den ST-LINK-Server der Kursversion. OpenOCD bleibt draußen, solange er nicht der Server ist. Der Ablauf ist: beide Images bauen oder als ungebaut kennzeichnen, Startadressen prüfen, beide laden, gemeinsam zurücksetzen, den Handshake durchlaufen lassen und die ersten drei Ausgaben erwarten. 1, 3 und 5 sind die Protokollvorhersage, kein vorhandenes Messergebnis. Der UART-Besitzer ist der Kern, dessen Retargeting wirklich schreibt. Breakpoints verändern Timeouts. Ein Speicherfenster des Debuggers ist nicht automatisch die Sicht des M7-Caches. Ohne Projekt bleibt der Lauf offen.
Ein kontrolliertes Fehlerexperiment dient danach der Ursachenanalyse.

Folie 53 / 55

Kontrolliertes Cachefehler-Experiment und Diagnose

Symptom	Prüfung	typische Ursache
M4 startet nicht	Stop, HSEM-ID	Boot, nicht Cache
falsche Adresse	beide Maps	Linker
Magic falsch	Speicherfenster	Init fehlt oder Zero läuft
M7 sieht alten Wert	CCR, MPU-Region	Cachezeile
Timeout	Zustand eingefroren	Ownership oder Barriere
falsches Image	ELF-Namen	Bank/Startadresse

GPIO oder UART-Status je Symptom, keine erfundene Messung.

Notizen 53 / 55

Die Tabelle trennt Hypothesen. Startet der M4 nicht, zuerst Stop und HSEM-ID, nicht den Cache. Falsche Adresse: beide Maps. Falsches Magic: Init oder Zero. Alter Wert beim M7: CCR und die MPU-Region. Ein Timeout beweist keine einzelne Ursache. HSEM ist hier das Bootsignal, ELF das geladene Image. Ein M4-Task in `osDelay` lässt andere Tasks laufen und beweist keinen eingefrorenen Kern. Die Fehlervariante ist ein separater Build mit nachgewiesenem D-Cache und Write-Back, S gleich 0, ohne Wartung. Ein kurzer Erfolg widerlegt das Risiko nicht. Zwischen den Varianten gibt es einen gemeinsamen Neustart. Laufzeitsperren mit HSEM kommen später dazu.

Folie 54 / 55

Ausblick: Tracing in Termin 7, HSEM in Termin 8

- Boot-HSEM ist heute erledigt
- Eine HSEM-Sperre für jeden Zugriff ist Termin 8
- HSEM ersetzt weder MPU noch DMB
- Ein lokaler FreeRTOS-Mutex sieht den M7 nicht

Notizen 54 / 55

Der Kursplan nennt Tracing für Termin 7 und HSEM als Laufzeitsperre samt Safety für Termin 8. „Nächste Woche HSEM“ wäre der falsche Verweis. HSEM ist heute das Bootsignal. Später kann sie Berechtigung oder Benachrichtigung koordinieren. Sie ersetzt weder die MPU-Attribute noch DMB. Ein FreeRTOS-Mutex gilt nur im M4-Kernel und sieht den M7 nicht. MPU bleibt die Speicherschutzeinheit des M7. Safety heißt hier die spätere Betrachtung sicherheitsbezogener Anforderungen, kein schon erbrachter Nachweis. RTOS und DMB bleiben die heutigen Werkzeuge des M4 und der Reihenfolge.
Zum Abschluss werden die Bausteine des Protokolls geordnet wiederholt.

Folie 55 / 55

Zusammenfassung des Tages

- AMP partitioniert Firmware, nicht automatisch die Zeit
- Boot: M4 Stop, M7 init, HSEM Release
- Shared: eine Adresse, M7 initialisiert. Reihenfolge: Payload schreiben, DMB, Zustand veröffentlichen; Zustand beobachten, DMB, Payload lesen
- Cache: beide Richtungen; MPU non-cacheable plus DMB plus ein berechtigter Payload-Schreiber je Phase
- Ausgabe 1, 3, 5 ist das Protokoll, nicht eine Messung

Notizen 55 / 55

AMP trennt die Firmware und isoliert die Zeit nicht automatisch. Der Boot ist M4 in Stop, M7 initialisiert, HSEM gibt frei. HSEM ist die Hardwaresemaphore, ELF je ein Image, MPU die nicht cachebare Region, IPC der Block, DMB die Barriere. Die Reihenfolge lautet: Payload schreiben, DMB, Zustand veröffentlichen. Die Gegenstelle beobachtet den Zustand, führt DMB aus und liest danach die Payload. „Ein Schreiber“ meint einen berechtigten Payload-Schreiber je Phase. Den Zustand beschreiben beide Kerne abwechselnd. Die erste Ausgabe ist 1, weil der M4 aus der initialen 0 eine 1 macht. 1, 3 und 5 messen keine Latenz.
Frage: Warum ersetzt HSEM nicht MPU und DMB? Eine Benachrichtigung koordiniert den Boot. Sie schreibt keine Cachezeile zurück und ordnet Payload und Zustand nicht.
Quellen bleiben DS12919, RM0399 der Projektrevision, AN5361, AN5617, AN4838 und AN4839, CMSIS-Cache und Intrinsics, GCC volatile und GNU ld zu NOLOAD. Offen ohne Projekt: Cube-Version, Map, UART-Pin, VOS und Tickfrequenz.
Der nächste Termin betrachtet, wie man solche Zeiten überhaupt sichtbar macht.